Internet of Things Course (420-521-VA)

Final Project
Phase 2

Due Date:

Sunday, October 26th, 2025

Team Members (Group 5)

Cristian Racila

Parth Patel

Trevor Obodoechina

Teacher:

Samad Rostampour

rostamps@vaniercollege.qc.ca

Table of Contents

Overview *3*

System Design *3*

Data Flow 3

Implementation *4*

Task Distribution *4*

Overview

Phase 2 of the IoT Smart Store project requires:

1. Two DHT11 temperature and humidity sensors connected to ESP32 boards.
2. MQTT-based data communication with a Raspberry Pi broker.
3. Live dashboard visualization using gauges and charts.
4. Temperature alert system that sends an email when a threshold is exceeded.
5. User interaction through email ("YES" to turn on the fan)
6. Fan activation via a DC motor, with live dashboard status updates.
7. Customizable temperature thresholds stored in a cloud database.

System Design

Each ESP32 board reads data from a DHT11 sensor and publishes the readings to an MQTT broker hosted on a Raspberry Pi.
The Raspberry Pi subscribes to both refrigerator topics (Frig1 and Frig2), processes incoming data, and communicates with the dashboard.
If a temperature threshold is exceeded, the system sends an email alert to the user. Upon receiving a "YES" response, the Pi triggers a fan (DC motor) via GPIO and updates the dashboard in real time.

The system is fully connected to a Supabase backend for live data storage, threshold configuration, and user management.

Data Flow

- Each ESP32 collects temperature and humidity data from its respective DHT11 sensor.
- Data is published to the MQTT broker on the Raspberry Pi under topics Frig1 and Frig2.
- The Raspberry Pi subscriber receives data and updates Supabase in real time.
- If temperature exceeds the stored threshold, the system:
    - Sends an email alert ("The current temperature is X°C. Would you like to turn on the fan?").
    - Monitors incoming email responses live.
    - If the response is YES, activates the fan (DC motor) and updates the dashboard.
- The dashboard displays:

    - Two gauges for temperature and humidity per fridge.
    - Live fan status.
- System notifications and modals for alerts

Implementation

- Microcontrollers: 2 × ESP32 boards with DHT11 sensors
- Broker: Raspberry Pi running Mosquitto MQTT
- Database: Supabase (replacing SQLite)
- Backend Logic: Python subscriber for MQTT + email monitoring + fan control
- Frontend: Flask web interface with Bootstrap, Chart.js, and custom UI animations
- Hardware: Raspberry Pi, DC motor with driver, breadboard, resistors, and wiring

- Repository: https://github.com/Crisguts/IoT-Semester-Project-

Task Distribution

Team Member	Task
Cristian	Implemented the DC motor control system tied to fan activation. Developed the temperature alert system that sends automated emails when a threshold is exceeded and monitors responses live. Integrated conditional logic so replying “YES” turns on the fan. Added a test button to simulate alert events and created animated fan UI elements.
Trevor	Completely designed the UI/UX using Bootstrap, Chart.js, and multiple visual tools. Built responsive modals, interactive gauges, and refined dashboard animations for a professional and intuitive interface. Proposed the use of Supabase for cloud data synchronization and ensured all visual elements updated dynamically with real-time readings.

Parth	Fully implemented the Supabase integration, migrating the project from SQLite to a real-time cloud database. Managed authentication and live data streams between the ESP32s, Raspberry Pi, and dashboard. Designed the ESP32 connectivity and ensured live readings were continuously stored and reflected in the UI. As a preventive adaptation, Parth created separate Arduino files for each ESP32 unit to simplify scalability and debugging. He also shared the initial Arduino setup files with the team to establish consistent communication templates for all microcontrollers.